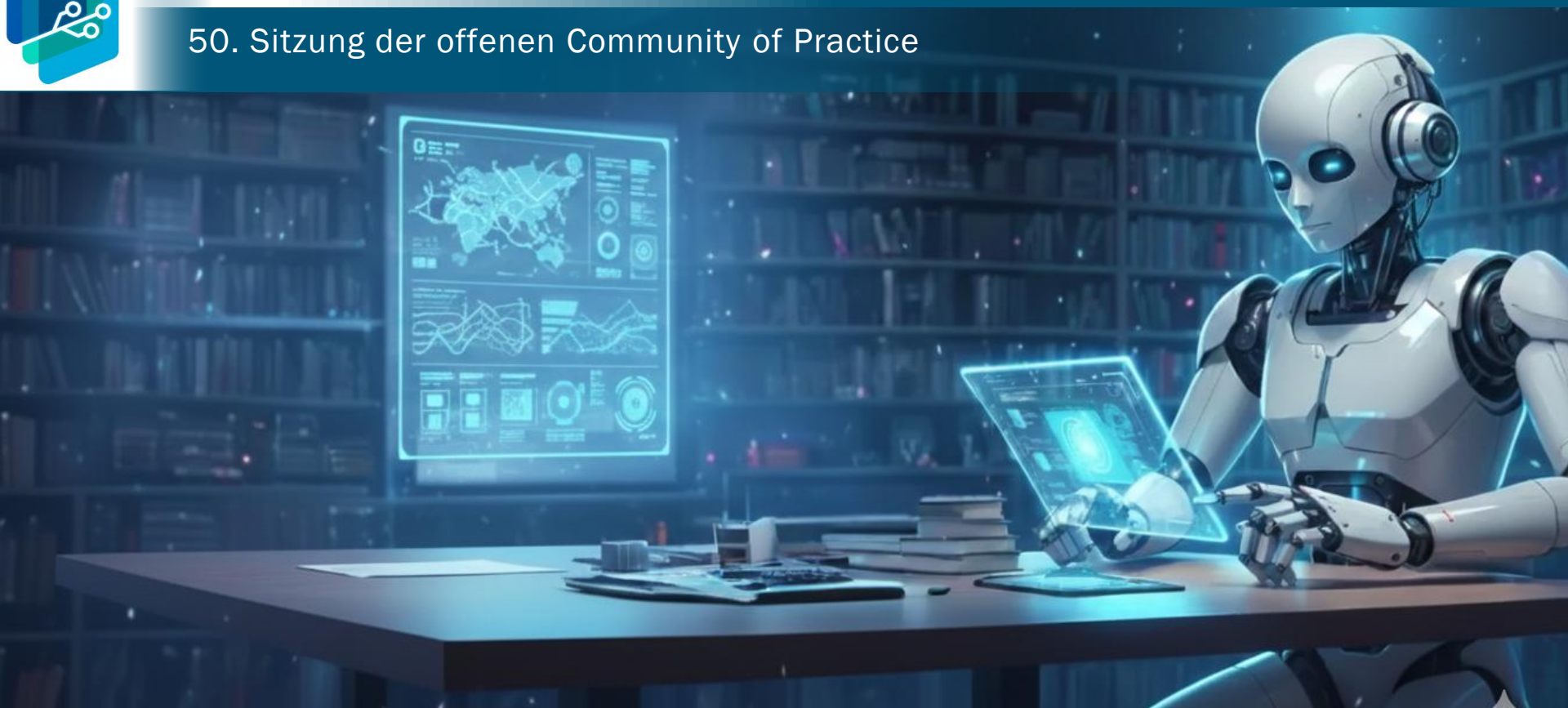




50. Sitzung der offenen Community of Practice



- **Update** (allgemein & persönlich)
- **Input:** „Kontinuum der KI-Autonomie - Tool & Function Calling im Wandel“
- **Diskussion**

Update (allgemein)



Hintergrund:

- Rechts abgebildet ist auszugweise das aktuelle LMArena-Leaderboard für die Kategorie „German“ mit Blick auf die besten open-weights-Modelle
- je 1 pro Modellfamilie, zzgl. kleine Modelle (<30B) sowie die Top-3 proprietären Modelle

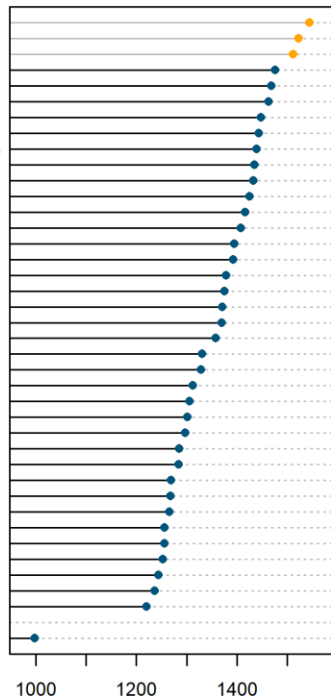
Bemerkenswert:

- Googles **Gemini-3-pro** wird abgelöst von **glm-5.3-max** (744B param. total, 40B active)
- Qwen3.8-27b** (27B param.) wird nun auch ausgewiesen und löst **Qwen3.5-27b** als effizientestes Open-Weights Modell um Längen ab
 - Hinweis: **Gemma-4** steht nach wie vor aus („overall“ liegt **gemma-4-26b-a4b** knapp und **gemma-4-31b** deutlich vor **Qwen3.8-27b**)

Arena Score German

selected models on lmarena.ai as of Sept 17, 2026

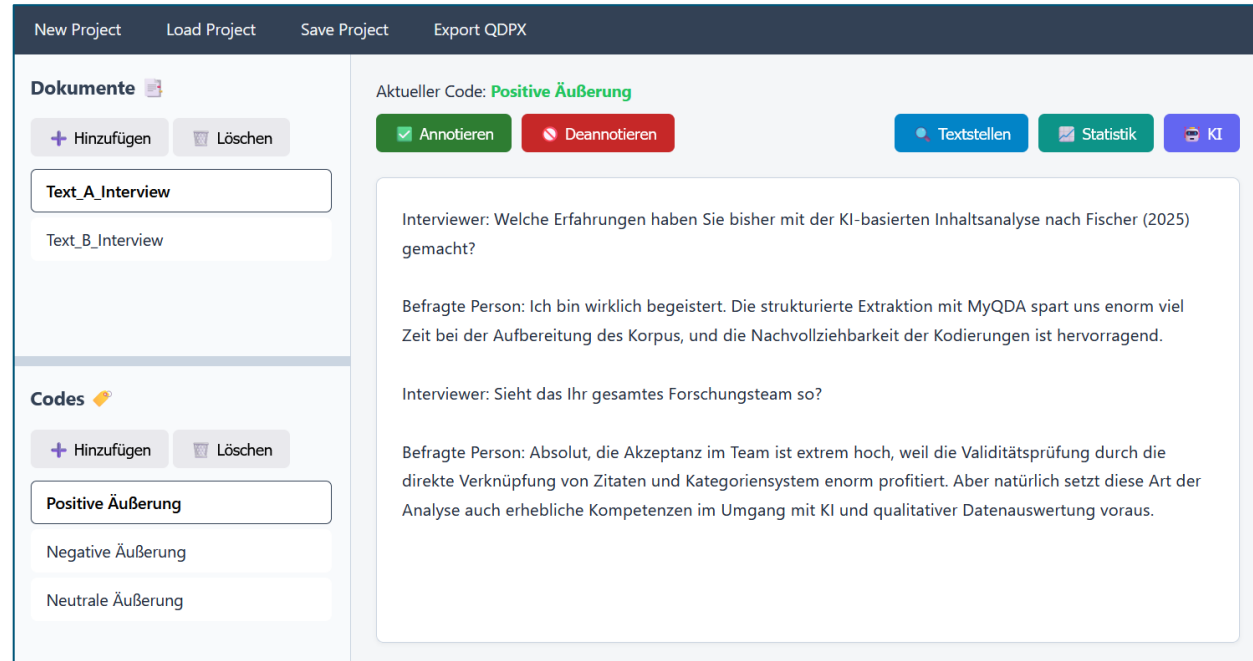
glm-5.3-max (Zai · MIT)
gemini-3-pro (Google · Proprietary)
claude-opus-4.6-high (Anthropic · Proprietary)
kimi-k3-max (Moonshot · Kimi K3 license)
deepseek-v4-pro (DeepSeek · MIT)
mimo-v2.5-pro (Xiaomi · MIT)
qwen3.8-27b (Alibaba · Apache 2.0)
qwen3.5-397b-a17b (Alibaba · Apache 2.0)
nvidia-nemotron-3-ultra-550b-a55b-nvfp4 (Nvidia · OpenMDW-1.1)
minimax-m3 (MiniMax · MiniMax Community License)
inkling (Thinky · Apache 2.0)
qwen3.235b-a22b-instruct-2507 (Alibaba · Apache 2.0)
Inkling Small (Thinky · Apache 2.0)
mistral-large-3 (Mistral · Apache 2.0)
qwen3.5-27b (Alibaba · Apache 2.0)
step-3.5-flash (StepFun · Apache 2.0)
longcat-flash-chat (Meituan · MIT)
trinity-large-thinking (Apache 2.0)
gemma-3-12b-it (Google · Gemma)
gemma-3-27b-it (Google · Gemma)
command-a-03-2025 (Cohere · CC-BY-NC-4.0)
llama-4-maverick-17b-128e-instruct (Meta · Llama 4)
gpt-oss-120b (OpenAI · Apache 2.0)
llama-4-scout-17b-16e-instruct (Meta · Llama)
qwq-32b (Alibaba · Apache 2.0)
olmo-3.1-32b-instruct (Ai2 · Apache 2.0)
mistral-small-3.1-24b-instruct-2503 (Mistral · Apache 2.0)
athene-70b-0725 (CC-BY-NC-4.0)
qwen-max-0919 (Alibaba · Qwen)
gemma-2-27b-it (Google · Gemma license)
gpt-oss-20b (OpenAI · Apache 2.0)
qwen2.5-72b-instruct (Alibaba · Qwen)
phi-4 (Microsoft · MIT)
mistral-small-24b-instruct-2501 (Mistral · Apache 2.0)
jamba-1.5-large (Jamba Open)
nemotron-4-340b-instruct (Nvidia · NVIDIA Open Model)
c4ai-aya-expense-32b (Cohere · CC-BY-NC-4.0)
qwen2.72b-instruct (Alibaba · Qianwen LICENSE)
...
qwen1.5-4b-chat (Alibaba · Qianwen LICENSE)



Update (persönlich) - MyQDA



- MyQDA erlaubt nicht nur *KI-basierte Analysen* (wir berichteten in [#49](#))
- MyQDA bietet nun auch by default die Möglichkeit einer *automatisierten Annotation* (für alle Analysevarianten) uvm.
- Ergebnisse lassen sich *manuell editieren* (auch vor der automatisieren Annotation 🤖)

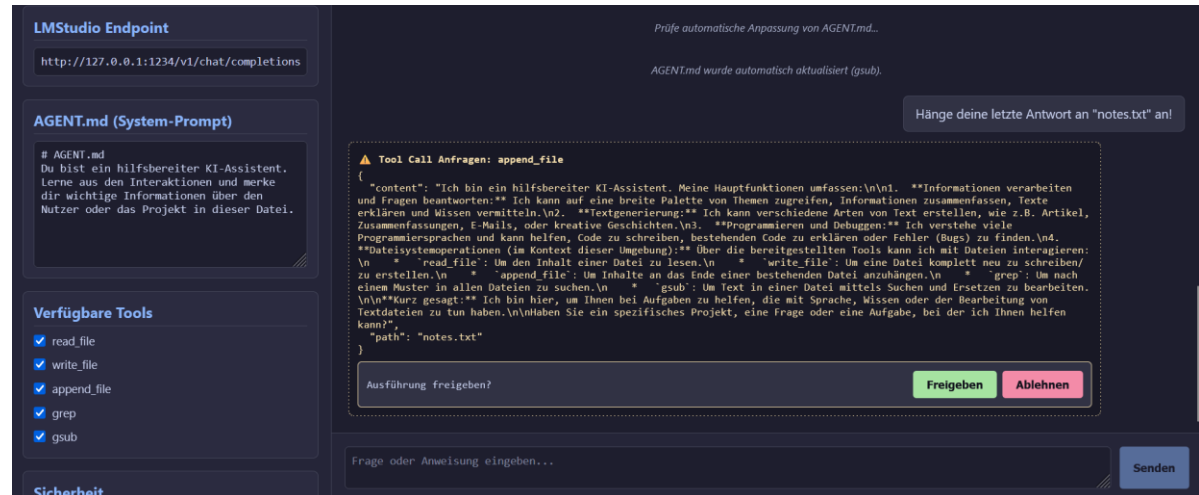


Quelle: <https://andreasfischer1985.github.io/code-snippets/html/MyQDA.html>

Kontinuum der Autonomie (vgl. Deci & Ryan '08)



- Tools wie MyQDA werden gezielt entwickelt, um KI-basierte Analysen durch Menschen und unter menschlicher Verantwortung zu optimieren
- Andere Systeme integrieren Tools in den Workflow:
 - (a) immer vor Anfragen (z.B. RAG),
 - (b) optional vor jeder Anfrage (i.S.v. IF-ELSE), oder
 - (c) iterativ bis zum gewünschten Output (i.S.v. WHILE-LOOP) – auch hier optional mit Freigaben
- Rechts exemplarisch ein MyAgentHarness (#49) (mit vom User bestimmten Tools und Freigabestufen sowie virt. Dateisystem als Sandbox) für flexible und sichere Einsätze (vgl. session #43)



Vgl. SDT: Edward L. Deci, & Richard M. Ryan (2008): Self-Determination Theory: A Macrotheory of Human Motivation, Development, and Health, S. 183. In: Canadian Psychology 49, 182–185.

Tool & Function Calling `26

Wie läuft Tool-Use 2026?

- 2023 hat man tool-calling noch im system prompt beschrieben ([#4](#), [#21](#))
- 2026 gibt es separate API-calls für effiziente tool-calls
 - Fragt man mit Parameter „tools“ kann die Antwort „function_call“ beinhalten
- Seit gpt-5.4 unterstützt OpenAI auch „tool-search“

Quellen

- <https://developers.openai.com/api/docs/guides/function-calling?api-mode=responses>
- <https://developers.openai.com/api/docs/guides/tools-tool-search>

```
from openai import OpenAI
import json
client = OpenAI()
tools = [{ "type": "function", "name": "get_horoscope", "description":
    "Get today's horoscope for an astrological sign.", "parameters": {
        "type": "object", "properties": { "sign": { "type": "string",
            "description": "An astrological sign like Taurus or Aquarius",
            }, }, "required": ["sign"], }, }, ]

def get_horoscope(sign):
    return f"{sign}: Next Tuesday you will befriend a baby otter."

input_list = [{"role": "user", "content":
    "What is my horoscope? I am an Aquarius."}]
response = client.responses.create(
    model="gpt-6-astra", tools=tools, input=input_list,)
input_list += response.output
for item in response.output:
    if item.type == "function_call":
        if item.name == "get_horoscope":
            # 3. Execute the function logic for get_horoscope
            sign = json.loads(item.arguments)["sign"]
            horoscope = get_horoscope(sign)
            # 4. Provide function call results to the model
            input_list.append({
                "type": "function_call_output",
                "call_id": item.call_id,
                "output": horoscope, })

print("Final input:")
print(input_list)
response = client.responses.create(
    model="gpt-6-astra",
    instructions="Respond only with a horoscope generated by a tool.",
    tools=tools, input=input_list, )
print("Final output:")
print(response.model_dump_json(indent=2))
print("\n" + response.output_text)
```

Vorsicht: Tool-calling im Detail Tool-abhängig

- LMStudio-Server & llama.cpp-Server nutzen **grammar-based sampling** (erlauben also bei der Inferenz nur Schema-konforme Tokens) 🤖
- vllm erfordert spezielle flags, weil ein nachgelagerter **Tool-Call-Parser** (z.B. pythonic, hermes, llama3_json, mistral, internlm2, openai) den LLM-Output ins gewünschte Format überführt:
 - enable-auto-tool-choice,
 - tool-call-parser pythonic
- Für vllm fährt man mitunter auch 2026 noch ebenso gut mit alten Workarounds:

```
curl https://fbb-vllm.cultus.dir/v1/chat/completions -H "Content-Type: application/json" -d
'{"model": "gemma-4-12b-it", "messages": [{"role": "system", "content": "Du hast Zugriff
auf folgendes Tool:\\nget_horoscope(sign: string) - Get today's horoscope for an astrological sign.\\n\\n
Wenn der User eine Frage stellt, die dieses Tool benötigt, antworte NUR mit dem JSON-Aufruf im
Format: {\\\\"name\\\\": \\\\"get_horoscope\\\\"}, \\\\"parameters\\\\": {\\\\"sign\\\\": \\\\"...\\\\"}}\\n"},
{"role": "user", "content": "Wie wird mein Tag heute? Ich bin Stier."}], "temperature": 0.0}'
```


The Tool Calling Flow

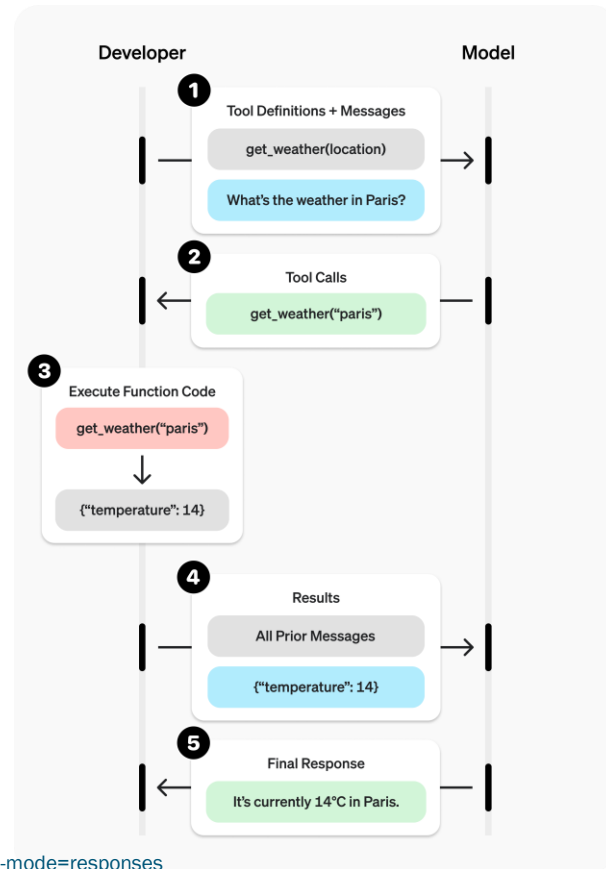


OpenAI's Definitions:

- **Tools:** functionality we give the model
- **Tool calls:** requests from the model to use tools
- **Tool call outputs:** output we generate for the model
- **Function:** a specific kind of tool, defined by a JSON schema.

Tool calling is a multi-step conversation between your application and a model via the OpenAI API. The tool calling flow has five high level steps:

1. Make a request to the model with tools it could call
2. Receive a tool call from the model
3. Execute code on the application side with input from the tool call
4. Make a second request to the model with the tool output
5. Receive a final response from the model (or more tool calls)



- Fragen?
- Anregungen?
- Erfahrungen?